CURSO DE GRADUAÇÃO EM ENGENHARIA DE SOFTWARE

Manutenção de software orientada por IA : Classificação de textos de relatos de erros, registrados em sistemas de helpdesk de apoio à implantação/produção de softwares educacionais, com a utilização de grandes modelos de linguagem.

Nome do aluno: Lilian Ramires Lima Castanha
Marcelo Nicolás Luján Villalba
Orientador: Profa DSc. Isabel Fernandes de Souza

Foz do Iguaçu/PR

Maio de 2026

Manutenção de software orientada por IA : Classificação de textos de relatos de erros, registrados em sistemas de helpdesk de apoio à implantação/produção de softwares educacionais, com a utilização de grandes modelos de linguagem.

Projeto de Pesquisa apresentado à componente curricular de Projeto Final de Curso I do Bacharelado em Engenharia de Software do Centro Universitário União das Américas - Uniamérica Descomplica como requisito parcial para obtenção do grau de Bacharel em Engenharia de Software.

O presente projeto de pesquisa encontra-se sob a orientação do Profa. DSc. Isabel Fernandes de Souza

Foz do Iguaçu/PR

Maio de 2026

SUMÁRIO

1 INTRODUÇÃO **4**
2 TEMA **4**
3 PROBLEMA **5**
4 HIPÓTESES **6**
5 OBJETIVO **7**
5.1 Objetivo Geral 7
5.2 Objetivos Específicos 7
6 JUSTIFICATIVA **8**
7 REFERENCIAL TEÓRICO **9**
8 METODOLOGIA **9**
9 CRONOGRAMA **9**
10 CUSTOS (opcional) **10**
11 RESULTADOS ESPERADOS **10**

RESUMO

Apresentar sucintamente uma introdução ao assunto, o objetivo da pesquisa, os materiais e a metodologia a serem empregados. O resumo deve ter entre 200 e 250 palavras.

Palavra-Chave:
No mínimo 3 e no máximo 5, diferente das contidas no título.

1. INTRODUÇÃO

A manutenção de software integra uma parte essencial do ciclo de vida dos sistemas, e costuma consumir bastante esforço técnico e financeiro depois que tudo já está funcionando e empregado após a implementação inicial (Pressman; Maxim, 2016; Sommerville, 2019). Em sistemas voltados para a educação — como ambientes virtuais de aprendizagem, plataformas de gestão acadêmica e soluções de ensino a distância — esse cuidado fica ainda mais crítico, qualquer falha técnica nessas plataformas pode impactar diretamente os processos pedagógicos, prejudicar a experiência do usuário e impossibilitar a continuidade das atividades institucionais.

Durante a implantação de softwares educacionais, é comum o surgimento de relatos de erros originários de diferentes perfis de usuários, como estudantes, docentes e gestores. Esses relatos, frequentemente descritos em linguagem natural e de forma não estruturada, precisam ser analisados, classificados e encaminhados adequadamente às equipes responsáveis. Esse processo, conhecido como triagem ou rastreamento de bugs, representa uma etapa essencial para a organização do fluxo de manutenção. Segundo Santos et al.,(2025), a diversidade de métodos, dados e métricas utilizadas na classificação de bugs torna o processo de análise e comparação dos resultados um desafio na área de engenharia de software.

Nos últimos anos, o avanço da Inteligência Artificial, especialmente dos Grandes Modelos de Linguagem (Large Language Models – LLMs), tem ampliado significativamente as possibilidades de automação de tarefas de análise textual e compreensão de código-fonte. Estudos recentes indicam que modelos baseados em deep learning apresentam resultados promissores na predição e classificação de defeitos de software, embora ainda enfrentem limitações relacionadas à representação semântica dos dados (Giray et al., 2023). Além disso, revisões recentes apontam o crescente uso de LLMs em atividades da Engenharia de Software, como análise de requisitos, geração de código e suporte à manutenção (Hou et al., 2024).

Nesse contexto, modelos como GPT-4, Gemini, LLaMA e CodeLlama demonstram capacidade de interpretar descrições de problemas e identificar padrões semânticos complexos, o que sugere potencial aplicação no apoio à triagem automatizada de bugs. A utilização adequada desses modelos depende, contudo, da construção de instruções eficazes, evidenciando a importância da chamada literacia de prompts no uso de sistemas baseados em IA, uma vez que a qualidade das respostas depende diretamente da formulação das instruções (Moura; Carvalho, 2023)

Apesar do crescimento das pesquisas sobre IA aplicada à Engenharia de Software, ainda se observa carência de estudos sistematizados que investiguem a aplicação de LLMs especificamente no processo de classificação de bugs em softwares educacionais. Tal lacuna evidencia a necessidade de aprofundamento teórico e

experimental nessa área, especialmente considerando as particularidades dos ambientes educacionais e a diversidade dos perfis de usuários envolvidos.
Dessa forma, o presente trabalho propõe investigar como os Large Language Models (LLMs) têm sido utilizados no rastreamento e gerenciamento de bugs em softwares educacionais, avaliando as abordagens relatadas na literatura, suas contribuições, limitações e potencial de aplicação prática. Além disso, busca-se propor diretrizes e desenvolver um protótipo preliminar que contribua para a otimização do processo de triagem de falhas.

2. TEMA

A classificação de erros de software em ambientes de implantação e/ou em produção continua sendo uma tarefa crítica. Portanto, o presente projeto de pesquisa, tem como foco o desenvolvimento de um estudo com a temática:

> *Manutenção de software orientada por IA : Classificação de textos de relatos de erros, registrados em sistemas de helpdesk de apoio à implantação/produção de softwares educacionais, com a utilização de grandes modelos de linguagem.*

3. PROBLEMA

A manutenção de software representa uma das fases mais críticas do ciclo de vida de sistemas, sendo responsável por parcela significativa do esforço total de desenvolvimento. Estudos clássicos indicam que atividades de manutenção podem consumir entre 60% e 80% dos recursos ao longo da vida útil de um software (Pressman; Maxim, 2016; Sommerville, 2019).

No domínio educacional, a manutenção assume relevância ainda maior. Plataformas como ambientes virtuais de aprendizagem e sistemas de gestão acadêmica precisam operar com alta disponibilidade, confiabilidade e usabilidade, atendendo a públicos heterogêneos compostos por estudantes, professores e gestores. Estudos recentes sobre qualidade de sistemas educacionais digitais destacam que esses fatores são determinantes para a efetividade das plataformas e para a experiência do usuário (Hou et al., 2024). A ocorrência de falhas nesses sistemas pode impactar diretamente processos pedagógicos e administrativos, comprometendo a continuidade das atividades institucionais educacionais.

Durante a fase de implantação de softwares educacionais, o volume de relatos de erros tende a aumentar, exigindo processos eficientes de triagem e encaminhamento de bugs.

Entretanto, quando realizados de forma manual, esses processos podem tornar-se demorados, suscetíveis a inconsistências e dependentes da experiência individual dos profissionais envolvidos (Pressman; Maxim, 2016).

Assim, a classificação de bugs de softwares, em especial, os de interesse desta pesquisa, ou seja, os educacionais, ainda é um problema que pode ter soluções que apliquem recursos tecnológicos contemporâneos a exemplo das LLMs. Portanto, o estudo organizado no presente projeto de pesquisa procurará responder às perguntas norteadoras:

É possível a *classificação de erros, relatados em período de implantação/produção, em softwares educacionais com o apoio de LLMs? Quais as evidências teóricas da efetividade das LLMs na classificação de erros?*

4. HIPÓTESES

Considerando o avanço das aplicações de LLMs na Engenharia de Software e sua capacidade de processar linguagem natural e analisar código, pressupõe-se que esses modelos possam oferecer ganhos relevantes no processo de rastreamento de bugs, especialmente em contextos que envolvem grande volume de relatos textuais de erro.

H1. A viabilidade da utilização de grandes modelos de linguagem na classificação de textos de relatos de erros, registrados em sistemas de helpdesk de apoio à implantação/produção de softwares educacionais.

5. OBJETIVO

5.1 Objetivo Geral

A presente pesquisa, organizada neste documento, procurará responder ao objetivo listado a seguir:

Classificar os textos de relatos de erros, com a utilização de grandes modelos de linguagem (Chat-gpt), registrados em sistemas de helpdesk de apoio à implantação/produção de softwares educacionais.

5.2 Objetivos Específicos

OE1 — Conceituar o processo de classificação de erros (bug tracking/bug triaging) no contexto da manutenção de softwares educacionais.

OE2 — Identificar, por meio de revisão integrativa da literatura, evidências da utilização de LLMs na classificação de erros relatados durante os períodos de implantação e produção em softwares educacionais.

OE3 — Mapear lacunas de pesquisa identificadas na literatura e propor uma API protótipo (LinkNix) para classificação e triagem de erros em softwares educacionais utilizando LLMs.

OE4 — Definir características relevantes para a classificação de erros e elaborar prompts a serem testados em, pelo menos, dois modelos de LLMs integrados à API LinkNix.

OE5 — Desenvolver a API LinkNix, incorporando um modelo de prompt para a classificação de textos de erros.

OE6 — Integrar a API LinkNix ao sistema de apoio HelpDesk JEDi Educa.

OE7 — Divulgar os resultados da pesquisa no formato de artigo científico em "Jornada de Iniciação Científica e de Empreendedorismo da Descomplica Uniamérica".

6. JUSTIFICATIVA

A crescente digitalização do setor educacional tem ampliado a dependência de plataformas tecnológicas para a realização de atividades acadêmicas, administrativas e pedagógicas. Nesse cenário, a estabilidade e a qualidade desses sistemas tornam-se fatores estratégicos para instituições de ensino e organizações de tecnologia educacional. Considerando que a manutenção representa parcela significativa do esforço total no ciclo de vida de software (Sommerville, 2019)torna-se essencial investigar mecanismos que possam otimizar processos relacionados à identificação e ao tratamento de falhas.

O rastreamento de bugs, etapa fundamental da manutenção, envolve a análise de relatos textuais, classificação de erros e priorização de correções. Trata-se de uma atividade que

exige interpretação semântica e raciocínio técnico, características que dialogam diretamente com as capacidades oferecidas por Large Language Models. Dessa forma, a aplicação desses modelos nesse contexto apresenta potencial inovador e merece investigação estruturada.

A relevância desta pesquisa fundamenta-se em três dimensões principais:

Do ponto de vista científico, observa-se que, embora existam estudos sobre LLMs na Engenharia de Software, a aplicação específica no domínio de softwares educacionais ainda é pouco explorada, configurando uma lacuna relevante na literatura.

Do ponto de vista prático, os resultados poderão fornecer subsídios para equipes de desenvolvimento de instituições de ensino, empresas de EdTech e projetos de software educacional, auxiliando na tomada de decisão quanto à adoção de ferramentas baseadas em Inteligência Artificial.

Do ponto de vista metodológico, o estudo contribui para o avanço de modelos de avaliação de LLMs aplicados à manutenção de software, área que se encontra em consolidação e demanda critérios mais sistemáticos de análise.

Assim, a pesquisa apresenta potencial de contribuir tanto para o aprofundamento teórico quanto para aplicações práticas no contexto da Engenharia de Software educacional.

7. REFERENCIAL TEÓRICO

O referencial teórico deste trabalho foi construído por meio de uma revisão teórica integrativa, método que permite reunir, analisar e sintetizar perspectivas diversas da literatura com o objetivo de consolidar o estado do conhecimento sobre um tema. O foco desta revisão recai sobre as evidências teóricas da efetividade do rastreamento de bugs com o apoio de LLMs em softwares educacionais, organizando o referencial em eixos temáticos interconectados.

7.1 Manutenção de Software: Conceitos e Desafios

A manutenção de software constitui uma das fases mais críticas e custosas do ciclo de vida de sistemas computacionais. Pressman e Maxim (2016) definem manutenção como o conjunto de atividades necessárias para manter um sistema de software operacional e adaptado às necessidades dos usuários após sua implantação. Sommerville (2019) complementa essa definição ao destacar que a manutenção representa tipicamente entre 60% e 80% dos custos totais de um projeto de software ao longo de sua vida útil.

Segundo Pressman e Maxim (2016), a manutenção de software pode ser classificada em quatro categorias principais: manutenção corretiva (correção de defeitos), manutenção adaptativa (adaptação a novos ambientes), manutenção perfectiva (melhoria de funcionalidades existentes) e manutenção preventiva (antecipação de problemas futuros). Entre essas categorias, a manutenção corretiva, que envolve o rastreamento e correção de bugs, representa uma parcela significativa do esforço total de manutenção.

Nas pesquisas de Sommerville (2019) há a ênfase aos desafios da manutenção de software, estes são amplificados pela complexidade crescente dos sistemas, pela escassez de documentação adequada e pela alta rotatividade de equipes de desenvolvimento. Esses fatores tornam o processo de identificação e correção de defeitos particularmente custoso e demorado, demandando novas abordagens e ferramentas que possam auxiliar os desenvolvedores nessa tarefa.

7.2 Rastreamento de Bugs: Definições e Abordagens

O rastreamento de bugs, também conhecido como *bug tracking* ou *bug triage*, é o processo sistemático de identificação, classificação, priorização e acompanhamento de defeitos em sistemas de software (Dr. Pushparani.MK et al., 2025). Segundo (Santos et al., 2025), esse processo é fundamental para garantir a qualidade do software e a satisfação dos usuários, uma vez que permite uma resposta organizada e eficiente aos problemas relatados.

Nos artigos de Pushparani et al. (2025) apresenta-se uma revisão histórica da evolução dos sistemas de rastreamento de bugs, destacando a transição de abordagens manuais e baseadas em planilhas para sistemas automatizados e integrados. Os autores argumentam que, com o crescimento da complexidade dos softwares e o aumento do volume de relatórios de bugs, tornou-se imprescindível o desenvolvimento de ferramentas que possam auxiliar na gestão eficiente desses defeitos.

O processo de rastreamento de bugs tipicamente envolve várias etapas: (1) submissão do relatório de bug pelos usuários ou testadores; (2) triagem inicial para verificar a validade e evitar duplicações; (3) classificação do bug quanto ao tipo, severidade e prioridade; (4) atribuição do bug a um desenvolvedor responsável; e (5) acompanhamento até a resolução e verificação da correção (Halit Yilmaz; Hakki Toroslu; Köksal, 2024)

Estudos recentes têm explorado o uso de técnicas baseadas em aprendizado profundo para estruturar e classificar relatórios de bugs de forma mais eficiente. Revisões sistemáticas indicam que modelos baseados em deep learning e representações semânticas apresentam melhor desempenho na identificação de padrões e similaridades entre relatos, contribuindo para processos de triagem mais precisos e escaláveis (Hou et al., 2024).

Trabalhos recentes têm explorado técnicas de aprendizado de máquina para automatizar diferentes aspectos do rastreamento de bugs. Tao et al. (2025) desenvolveram o sistema SCL-BT (*Structural Contrastive Learning Based Automatic Bug Triaging*), que utiliza aprendizado contrastivo estrutural para realizar a triagem automática de bugs com alta precisão. (Halit Yilmaz; Hakki Toroslu; Köksal, 2024) apresentam um estudo comparativo de diferentes paradigmas de aprendizado aplicados à detecção de prioridade de bugs, demonstrando que abordagens baseadas em *deep learning* superam métodos tradicionais em diversas métricas de desempenho.

Shao, Xiao e Yu (2026) exploram uma abordagem linguística para a identificação de bugs de concorrência, demonstrando que padrões textuais específicos nos relatórios podem indicar a presença desse tipo particular de defeito. Abordagens contemporâneas têm focado na otimização do processo de triagem e resolução de bugs por meio de técnicas avançadas de aprendizado de máquina, destacando ganhos em eficiência e redução do tempo de correção, especialmente quando integradas a modelos de linguagem e análise semântica dos dados.

7.3 Grandes Modelos de Linguagem (LLMs): Fundamentos

Os Grandes Modelos de Linguagem (*Large Language Models* - LLMs) representam uma das inovações mais significativas no campo da inteligência artificial nas últimas décadas. Esses modelos são sistemas de processamento de linguagem natural treinados em vastas quantidades de dados textuais, capazes de compreender, gerar e manipular linguagem humana de forma sofisticada (Hou et al., 2024).

Moura e Carvalho (2023) destacam a importância da literacia de *prompts* para potencializar o uso efetivo da inteligência artificial na educação. Os autores argumentam que a capacidade de formular instruções claras e contextualizadas (*prompts*) é fundamental para extrair o máximo potencial dos LLMs, permitindo que esses sistemas forneçam respostas mais precisas e relevantes às necessidades específicas dos usuários.

Simão et al. (2025) expandem essa discussão ao explorar o conceito de Inteligência Artificial Agêntica na Educação, onde LLMs não apenas respondem a consultas, mas atuam de forma proativa e autônoma para auxiliar em processos educacionais complexos. Essa abordagem agêntica tem implicações significativas para a aplicação de LLMs em contextos técnicos, como o rastreamento automatizado de bugs.

A arquitetura dos LLMs contemporâneos baseia-se principalmente em transformadores (*transformers*), uma estrutura neural que permite o processamento paralelo de sequências de texto e a captura de dependências de longo alcance. Esses modelos são pré-treinados em grandes corpus de dados através de tarefas como predição de próxima palavra, desenvolvendo uma compreensão profunda de padrões linguísticos, conhecimento factual e raciocínio lógico.

7.4 LLMs Aplicados à Engenharia de Software

A aplicação de Grandes Modelos de Linguagem à Engenharia de Software representa uma área de pesquisa em rápida expansão. (Hou et al., 2024) conduziram revisões sistemáticas da literatura que mapeiam o estado da arte dessa intersecção, identificando diversas aplicações promissoras: geração automática de código, documentação de software, detecção de vulnerabilidades de segurança, e, crucialmente para este trabalho, análise e rastreamento de bugs.

Giray (2022) fornece uma análise crítica sobre o uso de *deep learning* na predição de defeitos de software, contextualizando os avanços dos LLMs dentro da evolução mais ampla das técnicas de

aprendizado de máquina aplicadas à qualidade de software. O autor demonstra que modelos baseados em aprendizado profundo superam consistentemente abordagens tradicionais em métricas de precisão e recall, especialmente quando treinados em grandes volumes de dados históricos de projetos.

Koyuncu (2026) apresenta um trabalho seminal sobre categorização de relatórios de bugs com granularidade fina utilizando LLMs e engenharia de *prompts*. O estudo empírico do autor demonstra que, através da formulação cuidadosa de instruções contextualizadas, é possível obter classificações de bugs significativamente mais precisas e detalhadas do que aquelas produzidas por sistemas tradicionais. Esse trabalho estabelece uma base metodológica importante para a aplicação de LLMs em tarefas específicas de rastreamento de bugs.

A revisão de Hou et al. (2023) identifica várias vantagens dos LLMs para a engenharia de software: (1) capacidade de compreender contexto técnico complexo expresso em linguagem natural; (2) habilidade de generalizar conhecimento entre diferentes domínios e projetos; (3) flexibilidade para se adaptar a novas tarefas através de *fine-tuning* ou *prompt engineering*; e (4) potencial para automatizar tarefas que tradicionalmente exigem expertise humana.

No entanto, Hou et al. (2024) também destacam desafios importantes: necessidade de grandes volumes de dados de treinamento, custos computacionais elevados, questões de interpretabilidade das decisões do modelo, e riscos de reprodução de vieses presentes nos dados de treinamento. Essas considerações são particularmente relevantes ao aplicar LLMs em contextos críticos como o rastreamento de bugs em software educacional.

7.5 Software Educacional: Características e Especificidades

No domínio educacional, a manutenção de software assume relevância ainda maior. Plataformas como ambientes virtuais de aprendizagem e sistemas de gestão acadêmica precisam operar com alta disponibilidade, confiabilidade e usabilidade, atendendo a públicos heterogêneos compostos por estudantes, professores e gestores. Estudos recentes sobre a qualidade de sistemas educacionais digitais destacam que esses fatores são determinantes para a efetividade das plataformas e para a experiência do usuário (Hou et al., 2024). Nesse contexto, a ocorrência de falhas pode impactar diretamente processos pedagógicos e administrativos, comprometendo a continuidade das atividades institucionais.
Os softwares educacionais apresentam características específicas que influenciam diretamente o processo de rastreamento de bugs:

1. **Diversidade de usuários:** Estudantes, professores e administradores com diferentes níveis de letramento digital interagem com o sistema, gerando relatórios de bugs com graus variados de precisão técnica.

2. **Contexto pedagógico:** Bugs podem estar relacionados não apenas a falhas técnicas, mas também a problemas de design instrucional ou inadequação de recursos educacionais.

3. **Sazonalidade de uso:** Períodos letivos concentram maior volume de uso e, consequentemente, maior incidência de bugs reportados, exigindo processos eficientes de triagem e priorização.

4. **Impacto na continuidade educacional:** Bugs críticos precisam ser corrigidos rapidamente para evitar interrupções no processo de ensino-aprendizagem, demandando mecanismos ágeis de identificação e resolução.

Essas especificidades tornam o rastreamento de bugs em software educacional particularmente desafiador, mas também oferecem oportunidades para a aplicação de tecnologias avançadas como LLMs, que podem processar linguagem natural de usuários não-técnicos e compreender contextos pedagógicos complexos.

7.6 Evidências Teóricas: LLMs no Rastreamento de Bugs em Software Educacional

A convergência entre LLMs, rastreamento de bugs e software educacional representa um campo emergente com forte potencial teórico e prático. A literatura analisada nesta revisão fornece evidências teóricas que sustentam a viabilidade e efetividade dessa aplicação.

Dessa forma, observa-se que diferentes abordagens vêm sendo propostas para automatizar o rastreamento e a classificação de bugs, com resultados progressivamente mais robustos. Enquanto técnicas tradicionais baseadas em aprendizado de máquina apresentaram avanços iniciais, estudos mais recentes indicam que modelos baseados em aprendizado profundo e, especialmente, os Large Language Models, oferecem maior capacidade de interpretação semântica e generalização.

Os trabalhos analisados convergem ao evidenciar que a utilização de LLMs pode melhorar significativamente a precisão na classificação de relatos de bugs, sobretudo em cenários que envolvem linguagem natural não estruturada. Além disso, a integração de técnicas como *prompt engineering* e representações estruturais dos dados tem se mostrado eficaz para potencializar o desempenho desses modelos.

Entretanto, apesar dos avanços, a literatura ainda apresenta limitações no que se refere à aplicação dessas abordagens em domínios específicos, como o de softwares educacionais. A maioria dos estudos concentra-se em contextos gerais de engenharia de software, sem considerar as particularidades desse domínio, como a heterogeneidade dos usuários e a natureza dos relatos de erros.

Diante disso, identifica-se uma lacuna de pesquisa relevante, relacionada à investigação da aplicabilidade dos LLMs na classificação de bugs em softwares educacionais, especialmente durante os períodos de implantação e produção. Essa lacuna justifica a realização do presente estudo, que busca contribuir para o avanço do conhecimento na área e para o desenvolvimento de soluções mais eficientes no processo de manutenção de software.

8. METODOLOGIA

Esse tópico deve conter a descrição detalhada da metodologia de pesquisa utilizada, bem como do método de análise dos resultados. É permitido que a seção seja dividida em subtópicos.

Justifique a escolha dos métodos, explicando sua adequação aos objetivos da pesquisa. Além disso:

- Apresente a classificação da pesquisa quanto aos objetivos, à natureza dos dados e os procedimentos técnicos;
- Definir (caso necessário), a população e a amostra, descrevendo os critérios de seleção, inclusão e exclusão;
- Especificar os instrumentos de coleta de dados, como questionários, entrevistas, observações, bases de dados;
- Detalhar os procedimentos de análise dos dados, incluindo técnicas estatísticas quantitativas e qualitativas.

9. CRONOGRAMA

É necessário indicar o cronograma de realização do trabalho, o que dependerá do tempo disponível para a realização da pesquisa. A pesquisa deve ser dividida em partes, com previsão do tempo necessário para passar de uma fase a outra. Algumas partes podem ser executadas simultaneamente, enquanto outras dependem das fases anteriores. Assim, o cronograma visa a distribuir o tempo total disponível para a realização da pesquisa, incluindo nesta divisão a elaboração do relatório final.

Exemplo: Segue uma sugestão, segundo Santos (2002).

ETAPAS	Jul-Ago/ XX	Set-Out /XX	Nov-Dez/ XX	Jan-Fev/ XX	Mar-Abr/ XX	Mai-Jun/ XX
xxx	X	X				
xxx		X				
xxx			X	X		
xxx					X	
xx						X
xxx						X

10. CUSTOS (opcional)

11. RESULTADOS ESPERADOS

Os resultados esperados são a forma concreta em que se espera alcançar os objetivos específicos, são as contribuições que a pesquisa pretende trazer, tanto em termos teóricos quanto práticos. Portanto, deve existir uma correspondência muito estreita entre os objetivos, métodos e hipóteses de pesquisa com os resultados esperados.

12. Descreva os resultados que se espera alcançar com a pesquisa;
13. Indique como os resultados podem contribuir para o avanço do conhecimento na área/tema de pesquisa;
14. Mencione os benefícios práticos que podem decorrer da pesquisa. Por exemplo, uma nova ferramenta disponível para o público X; Um novo processo de trabalho, entre outros;
15. Relacione os resultados esperados com os objetivos e hipóteses de pesquisa.

REFERÊNCIAS

DR. PUSHPARANI.MK *et al.* The Evolution of Bug Tracking Systems: A Review of Advancements in Software Quality. **International Research Journal on Advanced Engineering and Management (IRJAEM)**, v. 3, n. 09, p. 2915–2918, 22 set. 2025.

GIRAY, Görkem *et al.* On the use of deep learning in software defect prediction. **Journal of Systems and Software**, v. 195, p. 111537, jan. 2023.

HALIT YILMAZ, Eyüp; HAKKI TOROSLU, İsmail; KÖKSAL, Ömer. A Comparative Study of Contemporary Learning Paradigms in Bug Report Priority Detection. **IEEE Access**, v. 12, p. 126577–126586, 2024.

HOU, Xinyi *et al.* Large Language Models for Software Engineering: A Systematic Literature Review. **ACM Transactions on Software Engineering and Methodology**, v. 33, n. 8, p. 1–79, 30 nov. 2024.

MOURA, Adelina; CARVALHO, Ana Amélia A. Literacia de Prompts para Potenciar o Uso da Inteligência Artificial na Educação. **RE@D - Revista de Educação a Distância e eLearning**, v. 6, n. 2, 2023.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.

SANTOS, Igor Ramsés Santos, Igor Ramsés Temóteo *et al.* Um Mapeamento Sistemático sobre Técnicas de Classificação e Categorização de Bugs em Software. **Revista Eletrônica de Iniciação Científica em Computação**, v. 23, p. 298–306, 31 dez. 2025.

SOMMERVILLE, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2019.

APÊNDICES

Apêndices são elementos pós-textuais, que complementam o projeto, **elaborados pelo próprio pesquisador**. Podemos trazer como exemplos os questionários, formulários de pesquisa, trechos de código-fonte, modelagem, diagramas.

ANEXOS

Os anexos são textos **elaborados por outras pessoas** e não pelo pesquisador. Como exemplo temos: mapas, plantas, documentos originais e fotografias tiradas por outros.

Só devem aparecer nos projetos de pesquisa, anexos considerados indispensáveis. Os apêndices localizam-se após as referências e os anexos, após os apêndices, se houver. Seus critérios de apresentação são:

- numerados individualmente com algarismos arábicos (quando há apenas um apêndice e/ou anexo, estes não devem ser numerados);
- cada apêndice e/ou anexo pode ser antecedido por uma página de rosto, na qual devem constar, escritos em letras maiúsculas estilo normal e centralizados na 13ª linha do texto, a palavra APÊNDICE e/ou ANEXO, seguida do número de ordem, de um hífen entre espaços correspondentes a uma letra, e do respectivo título;
- cada anexo e/ou apêndice inicia-se em página distinta;
- na página de rosto do anexo recomenda-se incluir elementos que identifiquem a fonte da qual foi extraído o conteúdo do anexo (autor, título,local e data).